

Algorithms and a platform for achieving org-level Super-Intelligence: Agent Vision, Sub-Agents, Recursively Self-Improving Recipes & Long-Term Memory Routers

Gunjan Gupta
Sanjiv Bishnoi

`gunjan@ommego.ai` `sanjiv@ommego.ai`

OmMeGo.AI

March 2026

Abstract

Agentic coding tools usually assemble context in an ad hoc way: the model sees pasted snippets, open files, or opaque retrieval, with little structure for team alignment, review, or persistence across branches and merge requests. We present SuperAgent, developed by OmMeGo.AI (<https://ommego.ai>) in partnership with Lightsphere AI (<https://lightsphereai.com>); it is offered on OmMeGo.AI with OmMeGo offerings (<https://ommego.com>). The productivity gains across both teams exceeded expectations and motivate releasing the stack broadly on OmMeGo.AI—formal measurement is in progress. Initial internal estimates—compared to Cursor with regular (non-SuperAgent) use¹—suggest on the order of ~50% improvement for developers new to the system and up to ~10x on complex coding tasks for advanced users, pending formal evaluation. Organizations publish catalog-backed sub-agents that declare an explicit input domain for assistants (analogous to a schema or proof domain), load topic-matched context through dual routers, and pair execution with recipes (structured Run / Verify steps) and layered delivery. A central idea is Agent Vision: seven explicit dimensions—from code diff and uncommitted changes through agent context, session history, build output, test output, and UI state—that generalize classic human–system supervision into a human–agent–system habit before planning or editing. We summarize formal sub-agents (OmMeGo.AI definition), an L0–L5 agentic work ladder (L1/L2 direct use through L5 meta-algorithm authoring), why layered agents differ from ordinary session-bound tooling, the domain router versus the L345 router, Router Builder indexing, artifact-level self-improvement without weight fine-tuning, and an end-to-end path (IDE-

¹Productivity baseline: Cursor with the same editor and model access as today, but without SuperAgent: only locally assembled unstructured context—ad hoc user prompts and snippets built arbitrarily by each developer or team, not using formal sub-agent markdown documents, dual routers, recipes, or a remote catalog—i.e. Cursor as teams typically use it before adopting the SuperAgent stack.

integrated client, Python CLI, /superagent, S3). We write for small and large teams in academia and industry alike—software organizations, domain experts, and process-led groups whose work is not primarily code—and argue for cross-organizational super-intelligence: durable, governed shared context instead of individual, session-bound agentic productivity patterns.

Introduction

Software organizations are adopting large language models for coding faster than shared governance models for those tools have matured. A single developer can hold effective context in chat; a team needs stable contracts for what the model may assume about the repo, tests, diffs, and release gates. Monolithic prompts and black-box retrieval make it hard to audit what the agent knew when it proposed a change.

Supervision lineage. Professional software practice already treats supervision as a closed loop between humans and systems: IDEs surface syntax and types; tests and harnesses return pass/fail signals; logs and metrics close the loop in production. That pattern is a two-way contract—the system shows state, the human (or policy) responds. Agentic assistants insert a third participant—the model—without removing the need for inspectable state. *Agent Vision* is the generalization we adopt: the same discipline as inspecting the relevant state before editing, extended so both people and models refresh human-agent-system views (diffs, builds, tests, UI, routed agent text) before acting. The same stance applies to non-developer domain experts who automate processes: their sub-agents play the role of a declared domain for the assistant, not an ad hoc chat transcript.

Affiliation and deployment. OmMeGo.AI (OmMeGo.AI 2026) (<https://ommego.ai>) and Lightsphere AI (Lightsphere AI 2026a) (<https://lightsphereai.com>) are partner companies that co-developed SuperAgent, with early use on Lightsphere AI internal workloads before general release on OmMeGo.AI. OmMeGo-branded offerings (Lightsphere AI 2026b) are described at <https://ommego.com>. The production /superagent endpoint is operated on OmMeGo.AI infrastructure.

Contributions. This paper: (1) formalizes *Agent Vision* as seven dimensions (A–G)—a human-agent-system situational checklist grounded in our IDE integration rules and router companion documents; (2) describes sub-agents as shared .md knowledge published under access control, and introduces *formal sub-agents* (OmMeGo.AI definition) as domain declarations that constrain what an LLM-assisted workflow may assume; (3) presents the L0–L5 agentic work ladder and contrasts layered org tooling with ordinary session-bound assistants; (4) explains dual routers—the domain router versus the L345 router—and the topic list to loadable summary to file load pattern, including Router Builder maintenance; (5) relates recipes to layered implementation, alpha and end-to-end (E2E) harness records (often JSON), merge-request discipline, and artifact-level improvement

without neural fine-tuning; (6) sketches SuperAgent services (session, provider secret key, catalog-backed publish). The narrative aligns with the public-facing SuperAgent overview in our internal design documentation (not cited here as a separate reference).

System sketch. A developer uses an IDE with workspace rules that force router context each turn. A Python CLI talks to the OmMeGo.AI `/superagent` endpoint with session and provider secret key; the server reads and writes routers and agent bodies in S3 under a provider id. Figure 1 situates these pieces.

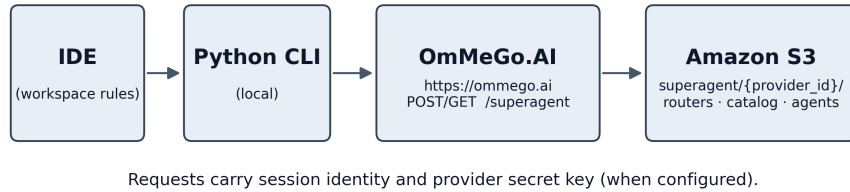


Figure 1: System context: IDE client, CLI, `/superagent`, S3.

Related work

This section situates our work for **agentic-AI** researchers, **industry** adopters at **any** scale, and readers who want **organizational** agent memory—including **process-led** teams and **domain experts** beyond classic **software** org charts. The narrative aligns with our **public-facing SuperAgent overview** in internal design documentation: *multi-level agent publishing*, **recipes** as executable operational knowledge, **Agent Vision** (structured situational perception, not a vague “context” dump), **interstitial layered development** with automated checks, and **dual routers** backed by long-term, organization-specific memory. We contrast *ad hoc* chat and monolithic prompts with **router-guided loading** of curated sub-agent text and with **decomposition** into sub-agents plus recipes—a practical pattern for teams that already rely on **Git**, **merge requests**, and **review gates**. Where classical tooling emphasized **human–system** feedback (IDEs, tests, telemetry), we keep that **spirit** explicit for **human–agent–system** triads so models do not become the **only** component without auditable state.

Tool use, reasoning loops, and instruction-tuned assistants

Large language models moved from static completion to **reasoning traces** and **tool invocation**. Chain-of-thought prompting showed that intermediate steps improve multi-step reasoning (Wei et al. 2022). **ReAct** interleaved natural-language reasoning with actions in an environment, providing a template for agent loops that many industrial stacks now mimic (Yao et al. 2022). **Toolformer** and surveys of **augmented language models** catalog how models

learn to call calculators, retrievers, and APIs (Schick et al. 2023; Mialon et al. 2023). **Reflexion** and **STaR** added self-critique and bootstrapped rationales for iterative improvement (Shinn et al. 2023; Zelikman et al. 2022). **InstructGPT**-style alignment made instruction-following assistants deployable in products (Ouyang et al. 2022), building on scaling trends (Brown et al. 2020; Vaswani et al. 2017). These lines of work motivate **agentic** coding assistants but typically assume a **single** model session and **ad hoc** context assembly—not an **organization-maintained** index of brains, routers, and **layered** verification tied to **merge workflows**.

Long contexts, retrieval, and the limits of “just add tokens”

Retrieval-augmented generation grounds generation in external evidence (Lewis et al. 2020). Empirical work on **long-context** models shows that performance can degrade when relevant facts sit in the **middle** of a packed prompt (Liu et al. 2024). Industry practice therefore oscillates between huge context windows and **RAG** pipelines. Our stance (developed within **OmMeGo** (Lightsphere AI 2026b) and operated on **OmMeGo.AI** (OmMeGo.AI 2026)) is complementary: we keep a **hierarchical topic index** over **your** agent and recipe corpus and load **small, task-matched** slices via **routers**, instead of pasting entire repositories or unstructured file dumps into every turn. Later in Related work, **RW-D3** contrasts *flat prompt stuffing* with *topic match to loadable summary to file load* at finer grain than **Figure 3** (router flow).

Multi-agent frameworks and software-building agents

Generative Agents popularized believable multi-agent simulations (Park et al. 2023). **MetaGPT**, **AutoGen**, and **ChatDev** package role specialization, conversation protocols, and software-oriented delegation for LLM agents (Hong et al. 2023; Wu et al. 2023; Qian et al. 2023). Surveys summarize autonomous agent architectures (Wang et al. 2024). **SWE-bench** and program-synthesis benchmarks measure whether models can patch real issues or synthesize programs (Jimenez et al. 2023; Austin et al. 2021). These systems emphasize **orchestration** and **benchmark scores**. We emphasize **governed editing** of shared **markdown brains**, **catalog-backed** publication, **dual routers** (broad discovery vs L345 agentic work), and **merge-request-aligned** recipes—a **process** fit for regulated teams, not only a leaderboard score.

Software engineering: transparency, coordination, and distributed work

Industrial strength requires **traceability** and **coordination**. **Social coding** on GitHub showed how visibility and discussion shape quality (Dabbish et al. 2012). **Coordination breakdowns** in large projects measurably hurt productivity and quality (Cataldo and Herbsleb 2013). **Distributed development** studies document communication and integration costs (Bird et al. 2011). **ML-for-code** surveys document how models interact with software artifacts at scale

(Allamanis and Sutton 2018). Our contribution is to **map** established social-technical practices onto **agent operations: merge-request sections, layered implementation, alpha/E2E recipes**, and **Agent Vision** dimensions that reviewers can check—so agentic work stays as **reviewable** as human code.

Contrasts for practitioners

Table RW-1 summarizes how a typical **IDE-integrated assistant** differs from a **router-indexed, catalog-backed** setup like SuperAgent. Table RW-2 positions prior research axes against our emphasis.

Table RW-1. Practical contrast (typical vs SuperAgent-style).

Dimension	Typical IDE chat (one session)	SuperAgent (org index + routers)
Primary context	Paste, search, open files	Routed sub-agent .md + catalog recipes
Knowledge ownership	Implicit / per developer	Catalog , access lists, publish path
Team alignment	Informal sharing	Shared agents, recipes , merge-request-style checklists
Forgetting / drift	Session loss, re-prompt	Persistent corpus; refresh on branch / merge-request events
Verification	Ad hoc tests	Layered E2E records (often JSON); alpha + merge-request gates

Table RW-2. Research axes vs our emphasis.

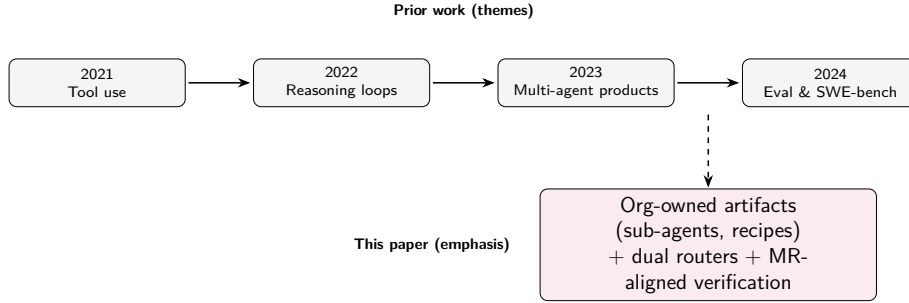
Axis	Representative prior work	Our emphasis (this paper)
Tool + reasoning loops	ReAct, Reflexion, Toolformer (Yao et al. 2022; Shinn et al. 2023; Schick et al. 2023)	Same loop idiom, plus router-selected domain text and recipes
Context scale	RAG, long-context studies (Lewis et al. 2020; Liu et al. 2024)	Hierarchical index + dual routers , not only longer prompts
Multi-agent products	MetaGPT, AutoGen, ChatDev (Hong et al. 2023; Wu et al. 2023; Qian et al. 2023)	Catalog ; L1/L2 use vs L345 authoring; merge-request process; Agent Vision
Code agents	SWE-bench, program synthesis (Jimenez et al. 2023; Austin et al. 2021)	Deployed story: CLI, OmMeGo.AI (OmMeGo.AI 2026), S3

Axis	Representative prior work	Our emphasis (this paper)
Human process	CSCW / empirical SE (Dabbish et al. 2012; Cataldo and Herbsleb 2013)	Maps review to layered delivery

Supplementary related-work diagrams (unnumbered)

The three **TikZ** panels below are **supplementary** visual anchors for this section: they use **unnumbered** captions (labels **RW-D1–RW-D3**) so they do **not** advance the main **Figure 1–Figure 7** sequence. They summarize **where** prior work clusters (tool use, multi-agent orchestration, flat retrieval) versus **our** emphasis on **router-indexed** sub-agents, **recipes**, and **merge-request** discipline. Readers who prefer prose only may skip them; the numbered figures in later sections carry the core architecture.

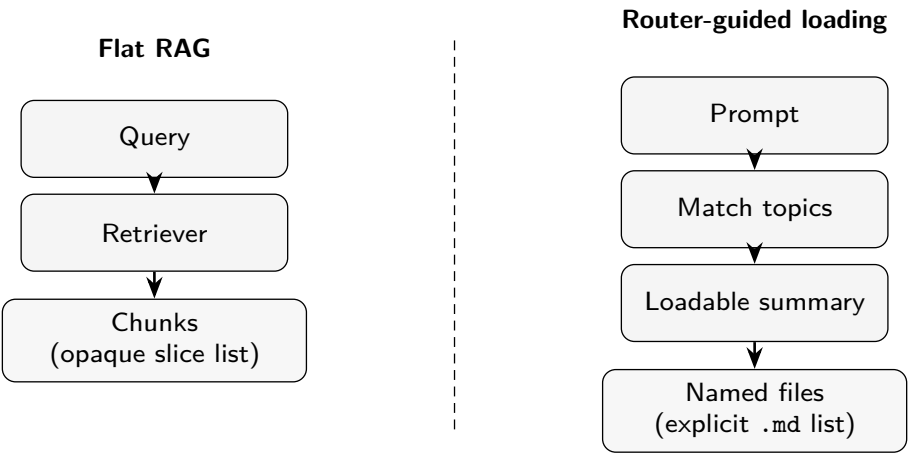
- **RW-D1.** Conceptual timeline: selected prior-work themes (tool use and reasoning loops) versus our dual-router + catalog stack.
- **RW-D2.** Positioning matrix: multi-agent orchestration research versus **process** / **merge-request** grounding versus the region our stack occupies.
- **RW-D3.** Contrasts **flat RAG** (query to a list of chunks) with **router flow** (prompt to topic summary to **named** context files)—the same contrast as **Figure 3**, at finer grain.



RW-D1: Conceptual timeline (selected prior-work themes) and our emphasis.

	More orchestration	Less orchestration
More MR/ process grounding	Multi-agent frameworks + workflow papers	Process / empirical SE
Less MR/ process grounding	Agentic benchmarks	SuperAgent routers + catalog + MR discipline

RW-D2: Conceptual positioning matrix (orchestration vs merge-request / process).



RW-D3: Flat RAG chunk lists vs router-guided loading of named context files (conceptual).

Agent Vision

We use **Agent Vision** to mean: before planning, coding, or tool use, the assistant decides which **situational views** the prompt requires and confirms they are **loaded and current**. If a required view is missing, the model is effectively blind—not from malice but from **wrong or incomplete perception** (see the internal design narrative on Agent Vision). The canonical definitions below match the **SuperAgent** technical reference (router architecture and Agent Vision dimensions) and **IDE** rule packs; we reproduce them as **Table 1** for the paper.

Dim	Description	When to include / typical sources
A Code diff	Changes to review or extend	vs default branch for merge request/release; vs named feature branch when user specifies; vs uncommitted for “what I edited”; none for green-field docs-only prompts.
B Uncommitted changes	Staged and unstaged working tree	Coding tasks, merge-request updates, “fix this”; exclude read-only or “ignore local changes”.
C Agent context	Sub-agent .md files selected by routers	Always via topic discovery: domain router for L1/L2 domain work; L345 router for L345 agent/recipe/router authoring; both when the task spans both.
D Session history	Recent turns and decisions	Follow-ups (“continue”, “same for Y”); exclude explicit fresh start.
E Run / build output	Server, build, deploy logs	Runtime failures, “build won’t start”; exclude when no run involved.
F Test output	pytest, Karma/Jest, al-pha/E2E results	Failing tests, interstitial verification; exclude before tests exist.
G UI view	Screenshots, browser/app state	UI bugs, layout; exclude pure API/backend design.

Table 1: Agent Vision dimensions (A–G): description and how to decide.

Check before coding: for each row, ask whether this prompt **requires** that view; if yes, load or refresh it. This discipline is what makes agent work **reviewable** in principle: reviewers can ask which dimensions were satisfied for a risky change.

Sub-agents

A **sub-agent** is a long-form **markdown** document: policies, APIs, recipes, and examples for a domain or process. It is intended as an **agentic brain** for a **sub-system**—current architecture, operational rules, and **near-term** improvements the team expects—not a one-off prompt file. Teams **publish** bodies to a **provider catalog** in **S3**; the server enforces **create vs update**, **creator / owner / developer** roles, and **access lists**. Locally, developers edit

copies; publication is a deliberate act, not silent sync. Sub-agents **complement** generic model weights: they encode **org-specific** rules, naming, and checklists. At scale, the catalog is the **shared brain**; routers choose **which** brains to load per turn, avoiding a single megaprompt.

Compared with **narrower** skill or **snippet** packs that ship a few procedures, a sub-agent in our stack carries **structured** sections, **cross-references**, and **recipes** that can **invoke** tools and APIs under review—closer to a **maintained** module of operational knowledge than to a loose bag of examples.

Dual routers

We maintain **two** router artifacts, each wired through an **IDE** workspace rule that applies **every** turn, both consulted with **non-overlapping** agent sets.

The **domain router** maps prompts to **domain** sub-agents in a **workspace-resident markdown corpus** (editor-specific layout; an **exclusion list** omits specialized docs that should not pollute the domain index). The **L345 router** maps prompts to **five L345 agents** only. Splitting indexes keeps **L1/L2** direct-use discovery separate from **L3–L5 authoring** context, reduces accidental loading of wrong corpora, and matches how teams think about “product work” versus “updating how we work.”

Algorithm (conceptual). (1) Extract goal from prompt and session. (2) Match **2–3 topics** from the router’s topic list. (3) Look up **sections** in the topic router cluster. (4) Emit a **loadable context summary** (which files, which sections). (5) Load only those slices before planning. **Figures 2 and 3** illustrate the split mental model and this flow; we do **not** paste full domain-router source here per paper scope.

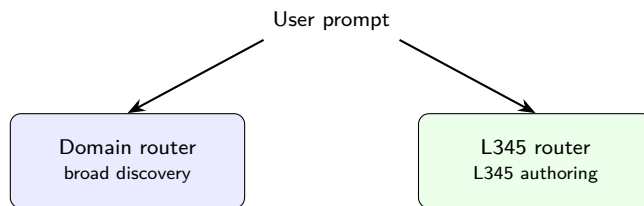


Figure 2: Dual-router mental model.

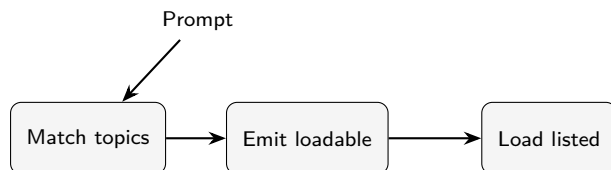


Figure 3: Router algorithm (conceptual): match topics; emit a loadable *summary*; load the listed *context files*.

The agentic work ladder (L0–L5)

We summarize **layered agent work** not as a **pricing tier** or a **job title**, but as a **depth-of-authoring** model for how work moves from **using** sub-agents to **improving** the agent ecosystem itself. **Figure 4** sketches the ladder; **Table 3** lists compact summaries. The **same person** may operate at different levels on different days; **most end-user work** concentrates in **L1** and **L2**, while **organizational learning** and paths toward **recursive self-improvement** at org scale align most clearly with **L3**, **L4**, and **L5**.

L1 — Creative sub-agent usage. Direct use for problems that are **not** fully predefined: the model must combine an **existing sub-agent brain** with **rich user context** to reach a solution that is **somewhat new**. This is **creative** use: it can be powerful but **less stable**, often needing multiple prompts.

L2 — Reliable sub-agent usage. Direct use for **complex** tasks the sub-agent **already** encodes well: **short prompts** can yield **long**, useful answers with **little or no** change to the core brain. This is the **dominant day-to-day** pattern for teams (features, bugs, tests, domain design under the **domain router**).

L3 — Sub-agent authoring. Indirect work that turns successful **L1** discoveries—from human steering or **agent-driven** exploration—into **durable** capability by **writing or updating** sub-agents and **recipes**.

L4 — Router structure authoring. Work on **topic indices**, **context bundles**, and **routing structure** that organize many sub-agents into a coherent **memory and loading** system (including **domain router** and related **Router Builder** outputs).

L5 — Meta-algorithm authoring. Work on **router-building** algorithms, **topic modeling**, and **memory refresh** logic that shapes **how the whole ecosystem learns** over time (e.g. multi-step paper or release orchestration that changes those mechanisms).

L0 (planned) — Minion layer. A **future** layer for **small**, tightly bounded **delegated** helpers—a **lowest execution tier** under the reasoning and authoring stack.

Why this is innovative: layered agents are different from ordinary AI tooling

The contrast is summarized in three columns below: ordinary tools, the OmMeGo layered model, and the result. The layout uses booktabs rules at the top and bottom only (no interior horizontal lines between rows).

Ordinary AI tools	OmMeGo layered model	Result
• Mostly answer user prompts • Little durable improvement • Knowledge stays in the session • Human has to re-discover solutions repeatedly	• L1/L2 solve work now • L3 converts successful new work into reusable agent knowledge • L4 organizes that knowledge into router structures • L5 improves how the system itself learns and refreshes memory	• Agent systems can grow from usage • Good outputs can become reusable structures • Organizations accumulate know-how instead of losing it • This is a path toward RSI / ASI-like self-improvement loops

Read left-to-right, the columns sketch a progression from answering immediate tasks, to capturing reusable knowledge, to organizing how that knowledge loads, to improving how the whole stack learns—the same arc the L0–L5 ladder formalizes in Table 3.

The domain router supports L1 and L2; the L345 router isolates L3–L5 meta-authoring inputs (five curated agents). We retain the internal L345 name for historical continuity; levels L3–L5 are the authoring bands that router serves. We use *RSI / ASI-like* in the informal sense of closed-loop improvement of org-owned artifacts and routing, not as a claim about benchmark AGI or unconstrained autonomous scaling.

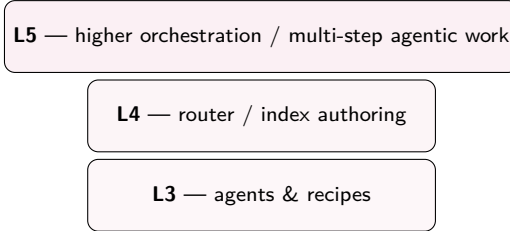


Figure 4: The agentic work ladder (L1–L5): creative vs. reliable direct use; authoring; router structure; meta-algorithms. **L0** (minion layer) is planned.

Beyond raw LLMs: domain constraints and formal sub-agents

Why do we need agents on top of large language models, beyond multi-step orchestration alone? In formal methods, proofs are stated relative to a domain of discourse; in databases, integrity and queries assume a schema that defines what tuples mean. Likewise, a learning system—including an LLM used as a component—behaves most reliably when its input domain is explicit: if the domain is ill-defined, behavior for out-of-domain prompts is inherently underspecified. Catalog-backed sub-agents are our practical way to declare that domain for organizational work: they state what the assistant is supposed to know for a slice of the system, which both sharpens answers inside the domain and makes durable improvement possible when those declarations are updated under review.

L2, L1, and L3 as a loop. L2 use is tied to fixed-domain knowledge already encoded in a sub-agent brain: the dominant reliable pattern (Table 3). L1 adds a small amount of creative extension beyond that fixed core; outputs can be less stable, but they stay anchored to the same constraints and context as the brain—typically more useful than asking a fully unconstrained general model that lacks that org-local contract. A human (or governed L3 process) then decides what is worth learning: for example, writing back a recipe after Agent Vision-guided repair of a supervised artifact (e.g., how to lay out tables next time), or promoting patterns from recent human-guided edits, corrections, or lightly novel tasks that went slightly past the brain’s prior envelope. That is the L2–L3 relationship the ladder summarizes: L2 executes inside a declared domain; L3 consolidates validated novelty back into reusable text.

Formal sub-agent (OmMeGo.AI definition)

We use **formal sub-agent** in a **product-specific** sense (not a generic mathematical definition). A **formal sub-agent** is an **OmMeGo.AI sub-agent** whose role and update rules are explicit enough to serve as a **domain declaration** for assistants and reviewers.

1. **Well-defined capability.** It is an **expert** in a **specific sub-system** for a **given domain**: scoped to **what it may assert** and **what it is responsible for** within that org and that slice of the stack—not an unconstrained general assistant.
2. **Well-defined storage and governance.** Its **brain** is **structured markdown** with predictable sections (policies, APIs, **recipes**, examples). **Higher-level authoring** on the **L3–L5 ladder**—especially **L3** edits to agents/recipes and **L4** work on **router structure**—states **how** that brain may change over time (who edits what, how topics are indexed, how cross-references stay valid). The **L2-scale** runtime use of the brain remains **separate** from those **meta** operations, but the two are **linked** through governed **publish** and **merge-request** habits.

3. **Routable and cross-referencable.** Real systems are **multi** sub-systems. Sub-agents therefore carry **stable labels** and **related-files** discipline so other agents (and humans) can **cite** and **traverse** knowledge **bidirectionally**; without that formality, **router-scale** loading and an inspectable **knowledge graph** across brains do not compose.
4. **Access-controlled writes and memory tiers.** Writes to published brains are **access-controlled** (owner / creator / developer roles on the **catalog**). We distinguish three **informal** memory layers: (a) **session-local** context that **evaporates** when the chat or IDE session ends unless promoted; (b) **long-term** knowledge in the **sub-agent** and **recipe** corpus, updated through **L3/L4/L5** authoring paths; (c) **medium-term** team state—merge requests, **TODO** lists, **alpha** evidence, **interstitial** logs, and other **in-flight** coordination—often **writable** by a **broader** class of contributors than who may **publish** a brain, yet still **reviewable** and **diffable** alongside code.

Feedback self-learning

Prior sections defined Agent Vision (Table 1), sub-agents, dual routers, the L0–L5 work ladder (Table 3), and formal sub-agents as domain declarations (above). We now make precise what we mean by *feedback self-learning* in this stack: not fine-tuning or weight updates to the base model, but closed-loop improvement of organization-owned artifacts—recipes and sub-agent text—driven by supervised outputs the assistant is required to inspect under Agent Vision (dimensions A–G), working-memory iteration within a session, and optional lightweight human nudges. At a high level, brains can improve after new tasks complete (via governed L3 edits), recipes can absorb task variations (human or policy-gated automation), and routers can re-index as the corpus grows—together yielding artifact-level RSI in the informal sense used in companion deck copy. We state the mechanism with a dedicated figure and caption so it is visible and reviewable; the claims stay conceptual and tooling-level, consistent with paper-facing naming and partnership disclaimers elsewhere.

Mechanism. A typical inner loop runs along a **recipe** (Run / Verify): the assistant **loads** Agent Vision and **examines** supervised artifacts—for example rendered LaTeX or TikZ figures, build logs, or test output—**iterates** until defects are resolved in **working memory**, and may receive **short human** feedback on taste or clarity. **Persistence** happens only when those lessons are **written back** into a **recipe** or **sub-agent** document; until then, improvements can be lost across sessions or branches. **Figure~5** encodes that pipeline and the **return** arc that supplies an **upgraded** baseline for the **next** invocation.

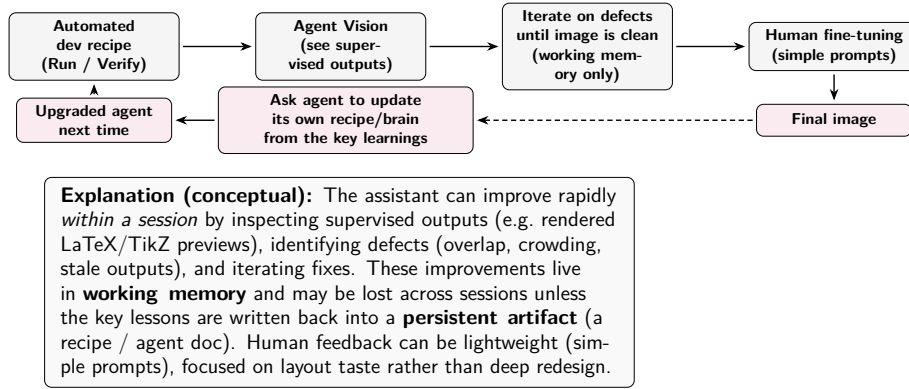


Figure 5: Feedback self-learning as **artifact** updates (conceptual). **Top row:** automated recipe work; **Agent Vision** over supervised outputs; iterative repair in **working memory** only; optional **human fine-tuning**. **Bottom row:** a finished supervised artifact (*e.g.* final image); consolidation into updated recipe/brain text (dashed arrows); an **upgraded** agent for the next session; and closure back to the recipe-driven entry point. The lower strip states the scope: rapid in-session improvement versus **persistent** artifacts—not neural fine-tuning.

Relation to deployment. The same habit ties to **layered** delivery and merge-request discipline: **Figure~6** summarizes how **recipes**, **alpha/E2E**, and merge-request **checkpoints** channel that persistence into reviewable team process—contrasting single-session tool use in much of the literature.

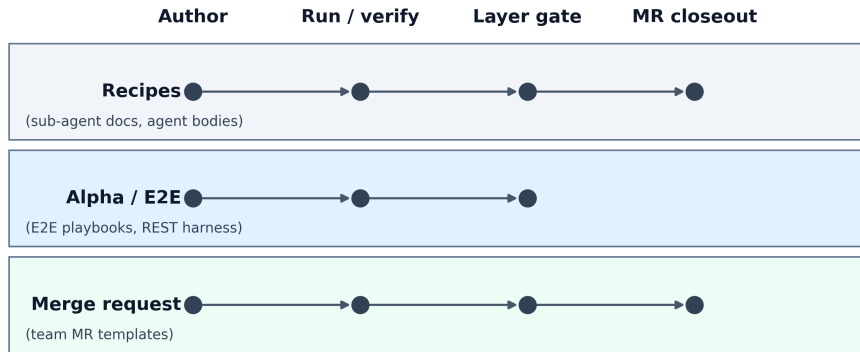


Figure 6: Recipes, layered implementation, alpha/E2E, and merge-request checkpoints: where structured SuperAgent work carries feedback into team process (compare to unstructured IDE chat in related work).

Recipes

Recipes are structured instructions agents follow: numbered sections, **Run** / **Verify** pairs, explicit paths and commands. In that sense they behave like **agentic programs**—**predictable** for their **core** steps, yet able to **extend** to many **variations** the authors did not enumerate, because the model can interpret natural-language branches **inside** the guardrails of the recipe and the hosting sub-agent. They sit **inside** agent docs and operational runbooks.

Authoring shape

Good recipes state **purpose**, use crisp bullets, avoid vague steps, and put commands in **fenced code blocks**. That format is shared with **alpha test case** prose so humans and agents can execute the same loop.

Layered delivery and E2E

Throughout, **E2E** means **end-to-end** checks that exercise services through their external interfaces (for example HTTP), not only isolated unit tests.

Layered implementation means: finish one **layer** (design slice) before the next; each layer may have **steps** and **E2E checkpoints** in **structured REST test catalogs** or product-specific JSON when behavior is testable. **Interstitial** work follows **documented E2E playbooks**: start server, run tests, fix, record stderr evidence, repeat. Recipes are the **glue** between merge-request text (“what we agreed to ship”) and **automated** checks.

Alpha, beta, and merge-request closeout

Alpha recipes target localhost and endpoint tests; **beta** recipes cover deploy and staging babysitting where applicable. **Team merge-request templates** expect **Layered Implementation, Details, Alpha** sections, and pointers to **E2E** evidence—so agent work stays parallel to human code review.

Publishing (experimental)

Some recipe classes may be **published** to object storage with version metadata; that path is **experimental** and not required for the SuperAgent core story. We mention it only to show how **recipes** can become first-class artifacts like agents. The **recipe / E2E / merge-request** cadence as one diagram appears in **Feedback self-learning** above.

Router Builder

Router Builder is the offline pipeline that regenerates topic frequency, per-agent topics, and **reverse indexes** from sub-agent sources. Domain and L345 pipelines write to **separate local index outputs** so generated topic lists never mix. The pipeline extracts short **topics** (2–5 words), builds **topic-to-agent**

maps, and feeds manual or automated updates to the **Topic Router** section of each router artifact. **Exclusion lists** keep infrastructure-only docs out of the **domain router** index. **Org managers** (or automation they approve) can also apply **incremental** edits to router text when the **learned** index should change without a full rebuild. This is how the corpus stays **maintainable** as dozens of agents grow.

Super Agent (end-to-end)

Super Agent names the deployed surface: authenticated requests to `/superagent` with an **action** (e.g. fetch routers, fetch agents, **publish_agent**, **add_access** / **remove_access**). The server reads the **router catalog** and agent objects under `superagent/{provider_id}/`, returns explicit **create denied** vs **update denied** messages when **access control** denies an operation, and uses **read-after-write** when returning catalog metadata so clients see what S3 stores. The **Python CLI** is the **IDE-friendly** client for the same API. Full REST tables are intentionally abbreviated in this paper; operators should rely on **published SuperAgent operator documentation** for authoritative action lists and error semantics.

Services (OmMeGo.AI)

OmMeGo.AI (OmMeGo.AI 2026) hosts the stack: users sign in (production traffic uses `https://ommego.ai`), obtain **session** identity, and (when configured) pass a **provider secret key** for SuperAgent calls. **Catalog access** is gated by published **access** lists per agent. **Release rationale**: early internal deployment showed a **strong productivity lift** versus **Cursor without SuperAgent** (unstructured local prompts and context only, no sub-agents or routers). We are opening access because that experience **outpaced** our initial expectations. **Productivity numbers** in the **Abstract** are **early internal measurements** against that **baseline**—not a formal benchmark or A/B study; **full methodology and reproducible evaluation are in progress**.

Discussion

SuperAgent is as much a **workflow** design as a **model** design. It speaks most directly to teams that already use **Git**, **merge requests**, and layered release notes, but the same **reviewable** habits transfer to **process-centric** and **non-software** organizations that still need **shared**, **auditable** agent knowledge. The architecture makes **context** and **procedures first-class** and **diffable**, which supports **cross-organizational** alignment—not only **individual** speedups. **Feedback self-learning** and the recipe/E2E figures above ground that loop; here we connect the habit to **merge-request** structure.

Merge requests, layered delivery, and Agent Vision

Organization merge-request guidelines define sections such as **Layered Implementation Sequence**, **Details / More Details**, **TODO**, **Alpha** (and related test staging), and **Evidence** expectations. **Sub-agent framework documentation** ties **global layer numbering**, per-layer **E2E checkpoints**, and **alpha** recipes to those merge-request habits. In practice:

- **Layered Implementation** lists ordered slices; each slice should be **green** before the next. That mirrors **Agent Vision** dimension **A** (what diff are we building on?) and **B** (what is currently dirty?).
- **Details** captures decisions and migration notes reviewers need—an offline analog to dimension **D** (session continuity across human reviewers).
- **Alpha** and endpoint JSON require **Run / Verify** evidence, aligning with dimensions **E** and **F** (build/test visibility).
- **UI-affecting** merge requests should attach screenshots or reproduction steps, echoing dimension **G**.

We do **not** claim a formal proof that every merge-request section maps one-to-one to A–G; **Table 6** suggests a **normative** mapping reviewers can adopt when auditing agent-assisted merge requests. **Figure~7** reminds readers to **refresh router-loaded context** after branch switches, rebases, or release cuts so **dimension C** stays current.

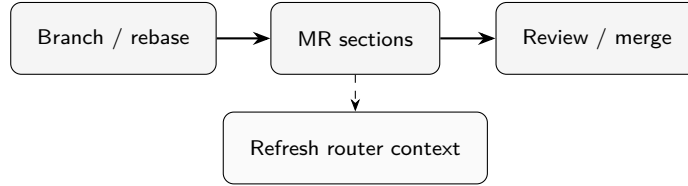


Figure 7: Merge-request lifecycle and when to refresh router context (after branch switch, rebase, or release cut).

Measurements

Quantitative claims and the **Cursor-without-SuperAgent** baseline are **defined and qualified** in the **Abstract** (with footnote ²) and in **Services**. Here we only fix **scope**: we report **directional, organization-internal** impressions—**not** a preregistered trial, independent benchmark, or A/B study. **Formal** task definitions, cohorts, logging, and **reproducible** methodology are **in progress**; treat cited ranges as **illustrative**, not guarantees for arbitrary teams, models, or repositories.

²Productivity baseline: Cursor with the same editor and model access as today, but without SuperAgent: only locally assembled unstructured context—ad hoc user prompts and snippets built arbitrarily by each developer or team, not using formal sub-agent markdown documents, dual routers, recipes, or a remote catalog—i.e. Cursor as teams typically use it before adopting the SuperAgent stack.

Limitations, ethics, and future work

Limitations

We describe a **production-style** architecture at the level needed for practitioners and agentic-AI readers; we do **not** give a complete formal specification of the **topic model**, **router** scoring, or **catalog** internals. **Long-term memory** and **hierarchical routing** are explained **conceptually**; algorithmic detail and scaling behavior belong in **companion** technical material. The mapping from **merge-request** sections to **Agent Vision** (Table 6) is **normative guidance**, not a proven one-to-one correspondence.

Ethics and safe deployment

Access control on the catalog reduces but does **not** eliminate risks of **prompt injection**, **cross-agent leakage**, or **misuse** of published agents; operators should enforce **least privilege** and review **publish** paths. **Closed-source** IDE assistants may adopt similar patterns; our emphasis remains on **visible**, **organization-owned** artifacts and **reviewable** workflows rather than on a single vendor stack.

Future work

Measurement and evaluation. Preregistered or clearly documented studies of **velocity**, **quality**, and **review burden** against the **Cursor-without-SuperAgent** baseline; **task suites** appropriate to regulated software work (not only synthetic coding benchmarks); and **third-party** or **multi-site** replication where feasible.

Modeling and systems. Tighter **formalization** of topic discovery, router **refresh** semantics, and behavior as the **agent corpus** grows; empirical analysis of **loadable-summary** quality and **missed-topic** failure modes.

Process and human factors. Quantitative and qualitative work on **merge-request** time, **defect** rates, and **reviewer** experience when agents follow **recipes** and **Agent Vision** under organizational policy.

Security and privacy. Dedicated analysis of **catalog** trust boundaries, **audit** logging, and **data residency** for providers that require it.

Dissemination. Two-column **arXiv** (or venue) layout with **final** figures and tables, venue-specific copy edit, and an optional **presentation** companion for teams centered on **merge-request** workflows—aligned with our release checklist but not required to use the deployed stack.

Conclusion

SuperAgent combines Agent Vision (A–G), catalog-backed sub-agents (including formal sub-agents as domain declarations), the L0–L5 ladder, dual routers,

recipes, Router Builder, and merge-request-aligned layering into one coherent pattern for agentic work at team and organizational scale. The goal is cross-organizational super-intelligence—durable, governed shared context—rather than isolated assistant tricks. It elevates the same habits software teams associate with diffs, tests, and merge gates (and analogous process checks elsewhere) so both humans and models operate against explicit, reviewable ground truth, and so successful work can graduate from answering prompts to improving the org-owned artifacts and indexes future turns rely on. Measurements, limitations, and future work are summarized in the preceding sections.

Acknowledgments

We thank Rodney Kinney³, Ulyana Krasnova, Bishwajit Dutta, and Sonali Walter⁴, Ashish Gupta, Rahul Sutradhar, and Aman Thakral⁵, Rohan Shinde, and Amol Kelkar for constructive feedback on this work. We also thank the OmMeGo.AI v2 Super Agent⁶.

References

Appendix: tables T2–T6

Table 1 (T1) appears in **Agent Vision** above. **Tables 2–6** below use booktabs rules, a single LaTeX caption per table (no duplicate “Table N” line in the body), and tabularx widths for the two-column layout pass.

Aspect	Domain router	L345 router
IDE workspace rule (every turn)	Domain-router rule pack	L345-router rule pack
Input corpus	Domain sub-agents (minus exclusion list)	Five L345 agents only
Typical tasks	L1/L2 direct use (features, bugs, domain design)	L3 agent/recipe edits, L4 router structure, L5 meta-algorithm workflows
Local index output	Separate build tree for domain topics	Separate build tree for L345 topics

Table 2: Domain router vs L345 router (summary).

³Allen Institute for AI.

⁴OmMeGo.AI.

⁵Lightsphere AI.

⁶OmMeGo.AI v2 Super Agent denotes the `omg_superagent.md` sub-agent at version 2. That sub-agent was used to build SuperAgent system version 3 and to draft this Manuscript; all diagrams and tables were produced with an RSI Image and table drawing recipe run within it. A hosted service on OmMeGo.AI is planned to open for registration by enterprise users in the near term.

Level	What it does	Typical user	Prompts	Output	Stability
L1	Creative direct use beyond fully predefined knowledge	Advanced end-user / developer	Medium-complex prompts	New solutions from existing brain + extra context	Can be unstable
L2	Direct use for complex but well-known tasks the sub-agent already knows	Most end-users	Short prompts, long answers	Reliable task execution	Highest day-to-day stability
L3	Write/change sub-agents and recipes based on successful new discoveries	Agent authors	Indirect/meta-prompts	New reusable knowledge	High leverage
L4	Build/update router topic indexes and memory structure	Meta-authors	Structured authoring prompts	Better routing and knowledge organization	System-level leverage
L5	Change router-building algorithms, topic modeling, memory refresh guidance	Core platform authors	Deep technical prompts	Changes how the whole system learns	Highest architectural leverage
<i>L0 (future)</i>	Minion layer for narrow delegated helper tasks	System + users	Very short tasking prompts	Small bounded execution	—

Table 3: The agentic work ladder (L0–L5): roles, prompt shape, outputs, and stability/leverage.

Category	Role	Examples
Authoring	How to write / structure recipes	Recipe of Recipes, merge-request voice
Operational alpha	Localhost E2E, server lifecycle	Documented alpha staging, automated endpoint harness
Operational beta	Staging / deploy babysitting	Beta recipes where shipped
Merge-request closeout	What to file before merge	Layered Implementation, Alpha evidence
Published (experimental)	Versioned blob + meta-data	KPI / future S3-backed recipes

Table 4: Recipe categories (informal).

Action (concept)	Purpose
Fetch routers	Return domain-router and L345-router document bodies for session
Get agents / catalog	List or fetch published agents per access policy
Publish agent	Create or update agent body + catalog row
Upload routers	Push new router markdown (manager workflows)
<code>add_access</code> / <code>remove_access</code>	Mutate catalog access lists with safety rules

Table 5: SuperAgent REST actions (abbreviated; see published operator documentation for full contract).

Merge-request section	Agent Vision dimensions emphasized
Layered Implementation	A, B, C (what we are changing; context loaded)
Details / More Details	D, A (decisions; baseline diff)
TODO vs Done	B (what remains local)
Alpha / E2E	E, F (runtime and test evidence)
UI or screenshot attachments	G
Security / secrets	C + policy (no secrets in text)

Table 6: Merge-request sections and Agent Vision (suggested review lens).

Allamanis, Miltiadis, and Charles Sutton. 2018. “A Survey of Machine Learning for Big Code and Naturalness.” *ACM Computing Surveys* 51 (4): 1–37.

Austin, Jacob, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, et al. 2021. “Program Synthesis with Large Language Models.” *arXiv Preprint arXiv:2108.07732*.

- Bird, Christian, Nachiappan Nagappan, Brendan Murphy, Harald Gall, and Premkumar T. Devanbu. 2011. “Does Distributed Development Affect Software Quality? An Empirical Case Study of Windows Vista.” *Communications of the ACM* 54 (8): 121–31.
- Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. “Language Models Are Few-Shot Learners.” *Advances in Neural Information Processing Systems* 33: 1877–1901.
- Cataldo, Marcelo, and James D. Herbsleb. 2013. “Coordination Breakdowns and Their Impact on Development Productivity and Software Quality: A Study of Modern Communication Collisions.” *Empirical Software Engineering* 18 (4): 643–83.
- Dabbish, Laura, Colleen Stuart, Jason Tsay, and James Herbsleb. 2012. “Social Coding in GitHub: Transparency and Collaboration in an Open Software Repository.” In *Proceedings of the Acm 2012 Conference on Computer Supported Cooperative Work*, 1277–86.
- Hong, Sirui, Xiawu Zhilin, Yuheng Chen, Jinlin Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, et al. 2023. “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.” *arXiv Preprint arXiv:2308.00352*.
- Jimenez, Carlos E., John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. “SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?” *arXiv Preprint arXiv:2310.06770*.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems* 33.
- Lightsphere AI. 2026a. “Lightsphere AI.” <https://lightsphereai.com>.
- . 2026b. “OmMeGo (Lightsphere AI).” <https://ommego.com>.
- Liu, Nelson F., Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. “Lost in the Middle: How Language Models Use Long Contexts.” *Transactions of the Association for Computational Linguistics* 12: 157–73.
- Mialon, Grégoire, Roberto Dessì, Maria Lomeli, and others. 2023. “Augmented Language Models: A Survey.” *arXiv Preprint arXiv:2302.07842*.
- OmMeGo.AI. 2026. “OmMeGo.AI.” <https://ommego.ai>.
- Ouyang, Long, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, et al. 2022. “Training Language Models to Follow Instructions with Human Feedback.” *Advances in Neural Information Processing Systems* 35: 27730–44.

- Park, Joon Sung, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. “Generative Agents: Interactive Simulacra of Human Behavior.” *arXiv Preprint arXiv:2304.03442*.
- Qian, Chen, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, et al. 2023. “Communicative Agents for Software Development.” *arXiv Preprint arXiv:2307.07924*.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. “Toolformer: Language Models Can Teach Themselves to Use Tools.” *arXiv Preprint arXiv:2302.04761*.
- Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. “Reflexion: Language Agents with Verbal Reinforcement Learning.” *arXiv Preprint arXiv:2303.11366*.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30.
- Wang, Lei, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jinyuan Zhang, Zhiyuan Chen, et al. 2024. “A Survey on Large Language Model Based Autonomous Agents.” *Frontiers of Computer Science* 18 (6): 186345.
- Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” *arXiv Preprint arXiv:2201.11903*.
- Wu, Qingyun, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibei Li, Erkang Zhu, Li Jiang, et al. 2023. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” *arXiv Preprint arXiv:2308.08155*.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. “ReAct: Synergizing Reasoning and Acting in Language Models.” *arXiv Preprint arXiv:2210.03629*.
- Zelikman, Eric, Yuhuai Wu, Jesse Mu, and Noah Goodman. 2022. “STaR: Bootstrapping Reasoning with Reasoning.” *Advances in Neural Information Processing Systems* 35: 15476–88.